

Tuning Confounds the Comparison: What a Controlled Study of Federated KANs for NetFlow Intrusion Detection Actually Shows

Phuc Hao Do, *Member, IEEE*, Van Long Nguyen, Tran Duc Le, and Truong Duy Dinh

Abstract—Kolmogorov–Arnold networks (KANs) have been proposed for federated intrusion detection on the promise that, at a matched parameter budget, they are more accurate and more stable than multilayer perceptrons under client heterogeneity. We test that promise on three standardised NetFlow-v2 datasets and it does not survive a controlled comparison. The advantage is confounded with how much tuning each architecture receives: sweeping the learning rate over six values, the +5.49 pp advantage we ourselves reported on NF-BoT-IoT-v2 falls to +0.76 pp when the parameter-matched MLP is given its own best rate, and to −0.54 pp ($p = 0.58$) under a nested protocol that selects the rate on ten seeds and reports on twenty never seen during selection. Replacing FedAvg with FedProx is a second, independent correction of the same kind, and the two compound: with both applied the MLP leads by 1.87 pp. Across four cells the advantage ranges from −0.49 to +6.23 pp with no consistent sign, and where clients are homogeneous the MLP is clearly ahead. The companion variance-reduction claim reverses on three cells of four. At matched parameters the KAN also costs $19.9\times$ more per-sample inference on CPU: parameter parity is not cost parity. Our contribution is the controlled comparison and the protocol that makes it controlled, released with 1,500 runs.

Index Terms—Federated Learning, Kolmogorov–Arnold Networks, IoT Security, Intrusion Detection, Non-IID, Robustness

I. INTRODUCTION

THE pervasive deployment of Internet-of-Things (IoT) devices across smart cities, industrial control, health-care, and autonomous transport has dramatically expanded the network attack surface. Distributed denial-of-service (DDoS) campaigns, botnets, reconnaissance scans, and other intrusions traverse the same edge gateways that carry legitimate IoT traffic, motivating the deployment of network intrusion detection systems (IDS) at the network edge [1].

Federated learning for IoT IDS. Centralised IDS pipelines [2], in which raw flow records are exfiltrated to a central server for model training, raise persistent privacy concerns and

conflict with regulations such as GDPR and HIPAA. Federated Learning (FL) [3], [4] offers a privacy-preserving alternative: each IoT gateway trains a local model on its own traffic and only the model updates are aggregated. A growing literature applies FL to IoT IDS, with notable contributions including DeepFed [5], which combined CNNs with gated recurrent units, and a wave of subsequent work on heterogeneity-robust aggregation, communication compression [6]–[8], and security-aware client selection.

The communication–robustness tension. Two pressures shape FL-IDS design at the edge. First, IoT links are bandwidth-constrained: a typical LoRaWAN device has a duty-cycled uplink budget on the order of kilobytes per hour. Second, client data is highly non-identical. The geographically distributed nature of IoT deployments means that one gateway may see almost only DDoS traffic while another sees primarily reconnaissance scans, a scenario commonly modelled by a Dirichlet partition with a small concentration parameter α [9]. Under such partitioning, vanilla FedAvg-MLP architectures are known to suffer slow convergence and, more pathologically, occasional catastrophic divergence in which a single unfortunate client–initialisation pair drags the global model far from the consensus optimum.

Kolmogorov–Arnold Networks (KANs). The recent introduction of KANs [10] replaces fixed neuron-wise activations with learnable edge-wise B-spline activations. Unlike a dense MLP weight matrix, where each scalar parameter modulates a global linear interaction, each spline coefficient in a KAN layer has *local* support on the grid by construction of the B-spline basis. This locality has been argued to provide better interpretability and parameter efficiency in physics-discovery and scientific-computing settings. In the IoT security domain specifically, KAN variants have been benchmarked for botnet attack classification in the centralised setting [11]; the question this paper asks is whether that behaviour survives federation under client heterogeneity.

KAN-FL: a recently busy research line. Within a year of [10], several works have integrated KANs into federated learning. F-KANs [12] ported the original KAN to a vanilla FedAvg framework on CIFAR-style benchmarks. [13] performed the first systematic non-IID evaluation of FedKAN, reporting modest gains over MLPs at matched parameter budgets on tabular data. [14] extended the comparison across multiple aggregation strategies (FedProx, SCAFFOLD), and

Phuc Hao Do is with the Department of Software Engineering, Danang Architecture University, Da Nang 550000, Vietnam (e-mail: haodp@dau.edu.vn).

Van Long Nguyen and Truong Duy Dinh are with the Faculty of Information Security, Posts and Telecommunications Institute of Technology, Ha Noi 10000, Vietnam (e-mail: longnv.b23at174@stu.ptit.edu.vn; duydt@ptit.edu.vn). Truong Duy Dinh is the corresponding author. This work has been sponsored by the scientific research from Posts and Telecommunications Institute of Technology, Vietnam

Tran Duc Le is with the Department of Mathematics, Statistics & Computer Science, University of Wisconsin-Stout, Menomonie, WI 54751, USA (e-mail: let@uwstout.edu).

[15] applied a Fed-KAN to traffic prediction. [16] contributed a medical-imaging benchmark and [17] an ECG-signal study. Most recently, [18] proposed CG-FKAN, which sparsifies the KAN grid coefficients to attack communication efficiency directly.

Positioning of this paper. The methodological observation “KAN-FL converges in fewer rounds and transmits fewer bytes than a parameter-heavy MLP” has therefore been published several times outside the IDS setting. We do *not* claim that observation as new. What is new in this paper is (i) a systematic empirical study of FedKAN *specifically* on the standardised NetFlow-v2 family of IDS datasets [19], (ii) a parameter-matched comparison protocol that resolves the unfair-baseline ambiguity present in much prior FedKAN work, and (iii) a refined narrative anchored in a convergence analysis: FedKAN’s advantage is not in mean accuracy but in worst-case robustness under extreme client heterogeneity, and this advantage cannot be replicated by simply scaling up an MLP baseline.

This failure mode has a literature. That a comparison can be decided by how well each side was tuned rather than by the methods themselves is not a new observation, though it appears not to have reached the federated-KAN line of work. It has been documented in language modelling [20], metric learning [21], recommender systems [22], generative models [23] and reinforcement learning [24], each time with the same shape: a proposed method is tuned carefully, the baseline is not, and the reported gain shrinks or vanishes when both receive the same budget. Related work on reporting practice argues for stating the search budget and the variance it induces [25], [26]. Our contribution is to carry that check into federated intrusion detection, where we find the same pattern in our own previously published numbers.

Headline finding. The advantage attributed to federated KANs in the recent literature, and in the first version of this manuscript, is confounded with how much hyper-parameter tuning each architecture receives. On NF-BoT-IoT-v2 binary detection under Dirichlet $\alpha = 0.1$, the parameter-matched MLP’s best learning rate is 10^{-1} , an order of magnitude away from the 10^{-2} shared by all architectures in the original comparison. Giving it that learning rate raises it by 5.39 pp and collapses the KAN advantage from +5.49 to +0.76 pp ($p = 0.74$, paired over ten seeds). The same sweep on three further cells puts the advantage between -0.49 and $+6.23$ pp with no consistent sign, so the effect is a property of the dataset, not of the architecture.

What the convergence analysis does and does not give. We retain a convergence analysis (Theorem 1) showing that the FedKAN error decomposes into a standard FedAvg term proportional to the heterogeneity factor Γ_α plus an additive spline-approximation bias of order $\mathcal{O}(G^{-(k+1)})$ that does not scale with Γ_α . We are explicit about its limits, because the first version of this paper was not. The bound is stated for the KAN alone, with no counterpart derived for the MLP, so it cannot support a comparison in either direction; and its spline term is non-negative, which if anything predicts a worse optimisation floor for the KAN rather than a better one. It is not cited anywhere in this paper as evidence for

an architectural advantage.

Contributions.

- **A negative result with a cause.** We show that the KAN-versus-MLP gap reported for federated intrusion detection is largely a tuning artefact, and we quantify it: on the cell that produced our own headline number, tuning the baseline removes 5.39 of the 5.49 pp gap, and a nested protocol that selects the rate on ten seeds and reports on twenty unseen ones puts the difference at -0.54 pp ($p = 0.58$). We also show the companion variance-reduction claim reverses sign on two of four cells.
- **A property that does generalise.** Across all three datasets, KAN is markedly less sensitive to the learning rate than the parameter-matched MLP, losing at most 0.65/1.58/2.53 pp from a bad choice within a decade-wide band against 3.53/6.38/6.72 pp, a factor of 2.7 to 5.4.
- **The price of that property.** On single-thread CPU at matched parameters, KAN inference costs $19.9\times$ more per sample and $37.9\times$ more per batch of 1000, and KAN transmits more bytes to reach a fixed accuracy target. Parameter parity is not cost parity, and a paper that matches parameters has not thereby matched cost.
- **A comparison protocol and the runs behind it.** Per-architecture learning-rate tuning, accuracy summarised over the final five rounds rather than the last round alone, and an explicit test of how far each conclusion moves when the summary statistic changes. All 1,500 runs, all configurations and the analysis scripts are released.

Organisation. Section II introduces the KAN layer formalism and the federated IDS optimisation problem. Section III details the FedKAN-IDS framework, including server and client algorithms. Section IV presents the four formal results that anchor our empirical narrative. Section V reports experimental protocol and results across architectures, partitions, and modes. Section VI discusses threats to validity and outlines next steps.

II. PRELIMINARIES AND PROBLEM FORMULATION

A. Kolmogorov–Arnold Network Layer

Kolmogorov–Arnold Networks (KANs) [10] replace the fixed neuron-wise activations of an MLP with *learnable* edge-wise activations parameterised as B-splines. Formally:

Definition 1 (KAN Layer). *Given input $\mathbf{x} \in \mathbb{R}^{n_{in}}$, grid size $G \in \mathbb{N}$, and B-spline order $k \in \mathbb{N}$, a KAN layer is the map $\Phi : \mathbb{R}^{n_{in}} \rightarrow \mathbb{R}^{n_{out}}$*

$$\Phi(\mathbf{x})_q = \sum_{p=1}^{n_{in}} \phi_{q,p}(x_p), \quad q = 1, \dots, n_{out}, \quad (1)$$

with each edge function

$$\phi_{q,p}(x) = w_{q,p}^{(b)} \sigma(x) + w_{q,p}^{(s)} \sum_{i=1}^{G+k} c_{q,p,i} B_i(x), \quad (2)$$

where $\sigma(\cdot) = \text{SiLU}(\cdot)$ is the residual base activation, $\{B_i\}_{i=1}^{G+k}$ is a B-spline basis of order k on a uniform grid of G points, and $\Theta_{KAN} = \{w_{q,p}^{(b)}, w_{q,p}^{(s)}, c_{q,p,i}\}$ are learnable.

By contrast, a fully-connected MLP layer with the same $(n_{\text{in}}, n_{\text{out}})$ employs a dense weight matrix $W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$, a bias $\mathbf{b} \in \mathbb{R}^{n_{\text{out}}}$, and a fixed scalar nonlinearity, so $\Theta_{\text{MLP}} = \{W, \mathbf{b}\}$. The architectural distinction relevant to our analysis is that each scalar weight in W controls a global linear interaction, whereas each spline coefficient $c_{q,p,i}$ has *local* support on the grid, by construction of the B-spline basis.

B. Communication-Constrained Federated IDS

Definition 2 (Communication-Constrained Federated IDS). Consider K IoT clients. Client C_k holds a private dataset $\mathcal{D}_k = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N_k}$ with feature vectors $\mathbf{x}_i \in \mathbb{R}^d$ and labels $y_i \in \{0, 1, \dots, c-1\}$, drawn independently from a per-client distribution $\mathcal{P}_k$. Heterogeneity across clients is modelled by drawing the per-class proportions of $\mathcal{P}_k$ from a Dirichlet distribution with concentration $\alpha > 0$ [9]: small α yields highly skewed $\mathcal{P}_k$ (extreme non-IID), while $\alpha \rightarrow \infty$ recovers the IID setting.

Given a model $f(\Theta; \cdot)$ and a per-client empirical loss

$$\mathcal{L}_k(\Theta) = \frac{1}{N_k} \sum_{i=1}^{N_k} \ell(f(\Theta; \mathbf{x}_i), y_i), \quad (3)$$

the federated IDS objective is

$$\min_{\Theta} F(\Theta) = \sum_{k=1}^K \frac{N_k}{N} \mathcal{L}_k(\Theta) \quad \text{s.t.} \quad \sum_{t=1}^T C_t \leq \mathcal{B}, \quad (4)$$

where $N = \sum_k N_k$, C_t is the per-round bytes summed across participating clients, and $\mathcal{B}$ is the deployment bandwidth budget.

In a synchronous FedAvg [3], [27] solver of (4), each round t samples a fraction $C \in (0, 1]$ of the clients, transmits the global parameters Θ_t to each, runs E local epochs of stochastic gradient descent, and aggregates the resulting local parameters $\{\Theta_{t+1}^k\}$ by data-weighted averaging.

C. Related Work

We position FedKAN-IDS at the intersection of three active research threads.

(i) **FL for IoT intrusion detection.** DeepFed [5] introduced a CNN-GRU architecture for federated industrial IDS, demonstrating that FL can match centralised detection rates while preserving data locality. Subsequent surveys [1] catalogued the design space of FL-IDS and identified two persistent bottlenecks: communication overhead on bandwidth-constrained gateways, and accuracy degradation under non-IID partitioning of network traffic. Robust FL methods [6] have addressed the latter by reducing the effective heterogeneity through compressed communication, but the underlying client architecture has remained, almost without exception, a dense MLP or CNN.

(ii) **Kolmogorov–Arnold Networks and their federation.** The KAN architecture of Liu *et al.* [10] replaces fixed neuron-wise activations with learnable B-spline edge activations, providing better interpretability and parameter efficiency on smooth scientific-computing benchmarks. Within a year of

that proposal, several works ported KAN to federated learning. F-KANs [12] integrated KAN into vanilla FedAvg on CIFAR-style image benchmarks. Sasse and de Farias [13] performed the first systematic non-IID evaluation of FedKAN on tabular data, reporting modest accuracy gains over MLP at matched parameter budgets and proposing the same parameter-matching protocol we adopt here. Ma *et al.* [14] extended the comparison across multiple aggregation strategies (Fed-Prox [28], SCAFFOLD [29]), and Zeydan *et al.* [15] applied a Fed-KAN to traffic prediction. Lee *et al.* [16] contributed a medical-imaging benchmark and Zeleke and Boichicchio [17] an ECG-signal study. Most recently, Yu *et al.* introduced CG-FKAN [18], which sparsifies KAN grid coefficients to attack communication efficiency directly.

The methodological observation underlying this body of work that FedKAN converges in fewer rounds and transmits fewer bytes than a parameter-heavy MLP has therefore been published several times outside the IDS setting. Our contribution is *not* to claim that observation as new. Rather, we contribute (a) the first systematic study of FedKAN on the standardised NetFlow-v2 family of IDS datasets [19], (b) a parameter-matched experimental protocol that resolves an unfair-baseline ambiguity present in prior FedKAN work, and (c) a controlled comparison showing that the advantage reported for FedKAN in this literature is confounded with per-architecture hyper-parameter tuning, together with the protocol needed to avoid that confound.

(iii) **Communication-efficient FL.** Our framing also intersects with the broader literature on communication-efficient FL. FedPAQ [7] reduces uplink bandwidth via periodic averaging plus stochastic quantisation; Top- k SGD [8] sparsifies gradient updates to the largest k entries; and [6] combined sparsification with non-IID-aware aggregation. CG-FKAN [18] can be viewed as a KAN-specific instantiation of this line. Our protocol does *not* apply any compression; we report all results in raw float32. As Section V makes explicit, under parameter parity FedKAN is in fact slightly *more* expensive to transmit than a comparable MLP, so the contribution of this work is not communication efficiency, and not an accuracy floor either: it is the controlled comparison itself.

III. THE FEDKAN-IDS FRAMEWORK

This section describes how Definitions 1 and 2 are instantiated in the FedKAN-IDS pipeline. Section III-A fixes the per-client model architecture and the global system flow (Fig. 1); Section III-B formalises the server orchestration (Algorithm 1); Section III-C details the client local optimisation (Algorithm 2).

A. System Architecture

The FedKAN-IDS pipeline alternates between four stages per communication round, depicted in Fig. 1: (1) the server initialises the global KAN with hyperparameters (n_h, G, k) ; (2) broadcasts the current parameter triple $\Theta_t = \{w_t^{(b)}, w_t^{(s)}, c_t\}$ to the participating clients; (3) each client performs local optimisation against its private NetFlow records; and (4)

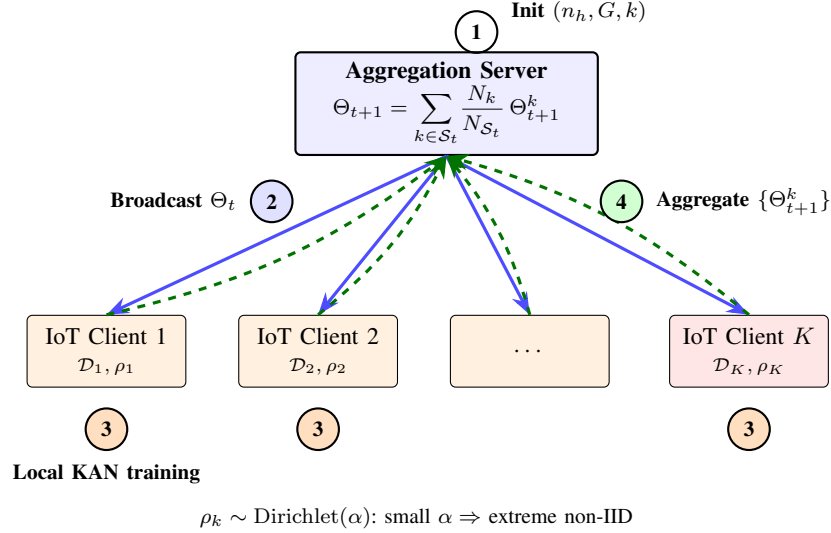


Fig. 1. Main flow of FedKAN-IDS over one communication round. (1) The server initialises the global KAN with hyperparameters (n_h, G, k) ; (2) broadcasts the current parameters $\Theta_t = \{w_t^{(b)}, w_t^{(s)}, c_t\}$; (3) each IoT client trains locally on its private $\mathcal{D}_k$ whose class proportions ρ_k are drawn from a Dirichlet with concentration α ; (4) the server computes the data-weighted average of returned local parameters element-wise on each of $\{w^{(b)}, w^{(s)}, c\}$. The structure of one client's KAN layer (stage 3 detail) is shown in Fig. 2.

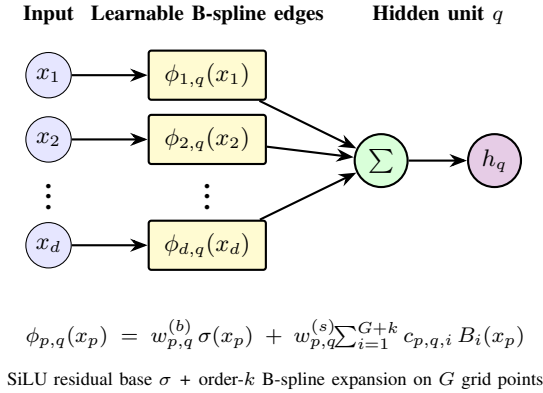


Fig. 2. Sub-flow: structure of one KAN layer producing hidden unit h_q from input vector $\mathbf{x} \in \mathbb{R}^d$. Each edge (p, q) carries an independent learnable activation $\phi_{p,q}$ parameterised as a SiLU residual base plus a B-spline expansion on a uniform grid of G points (order k). The full client model stacks an input layer ($d \rightarrow n_h$) and an output layer ($n_h \rightarrow c$). The B-spline coefficients $c_{p,q,i}$ have local support.

the server aggregates returned parameters by data-weighted averaging.

Each IoT client C_k instantiates a single-hidden-layer KAN

$$f(\Theta; \mathbf{x}) = \Phi^{(\text{out})}(\Phi^{(\text{hid})}(\mathbf{x})), \quad (5)$$

where $\Phi^{(\text{hid})} : \mathbb{R}^d \rightarrow \mathbb{R}^{n_h}$ and $\Phi^{(\text{out})} : \mathbb{R}^{n_h} \rightarrow \mathbb{R}^c$ are KAN layers (Definition 1) with shared grid size G and order k . The model output is passed through a softmax to produce class probabilities; we train with cross-entropy loss. The client parameter set $\Theta = \{w^{(b)}, w^{(s)}, c\}$ comprises three structurally distinct sub-tensors which the server aggregates element-wise.

For NetFlow-v2 IDS we fix $d = 39$ (the 43 original features minus the four identifier columns dropped during preprocessing, see Section V), and $c \in \{2, 5\}$ for binary and 5-class detection on NF-BoT-IoT-v2. Throughout, our

Algorithm 1 FedKAN-IDS Server Orchestration

Require: Number of clients K , sampling fraction C , rounds T , KAN hyperparameters (n_h, G, k) .

Ensure: Trained global parameters Θ_T .

- 1: Initialise $\Theta_0 = \{w_0^{(b)}, w_0^{(s)}, c_0\}$
- 2: **for** $t = 0, 1, \dots, T - 1$ **do**
- 3: $m \leftarrow \max(\lceil C \cdot K \rceil, 1)$
- 4: $\mathcal{S}_t \leftarrow$ random subset of m clients from $\{1, \dots, K\}$
- 5: **for each** client $k \in \mathcal{S}_t$ **in parallel do**
- 6: $\Theta_{t+1}^k \leftarrow \text{CLIENTUPDATE}(k, \Theta_t)$
- 7: **end for**
- 8: $\Theta_{t+1} \leftarrow \sum_{k \in \mathcal{S}_t} \frac{N_k}{\sum_{j \in \mathcal{S}_t} N_j} \Theta_{t+1}^k$ $\triangleright$ element-wise on each of $w^{(b)}, w^{(s)}, c$
- 9: **end for**
- 10: **return** Θ_T

default architecture is $n_h = 8$, $G = 5$, $k = 3$, giving $|\Theta| \approx 3,300$ parameters, well within the TinyML envelope of microcontroller-class IoT gateways.

B. Server Orchestration

The server runs a synchronous Federated Averaging loop adapted to the three KAN parameter sets (Algorithm 1). Each round, a fraction $C \in (0, 1]$ of the K clients is selected; in our experiments $C = 1$ (full participation), but the algorithm and analysis remain valid for $C < 1$. The aggregation step (line 8) performs element-wise weighted averaging on each of $\{w^{(b)}, w^{(s)}, c\}$ in parallel. Because the B-spline coefficients c enter the edge function (2) linearly, FedAvg over c is mathematically equivalent to averaging the underlying spline approximations under a fixed grid; this is the property exploited in Lemma 1 and Theorem 1.

Algorithm 2 FedKAN-IDS CLIENTUPDATE

Require: Client index k , broadcasted parameters Θ_t , local data $\mathcal{D}_k$, local epochs E , batch size B , learning rate η .

Ensure: Updated local parameters Θ^k .

```

1:  $\Theta^k \leftarrow \Theta_t$ 
2: partition  $\mathcal{D}_k$  into batches of size  $B$ 
3: for epoch  $e = 1, \dots, E$  do
4:   for minibatch  $(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_k$  do
5:     Forward: for each KAN edge  $(q, p)$ ,

$$\phi_{q,p}(x_p) = w_{q,p}^{(b)} \sigma(x_p) + w_{q,p}^{(s)} \sum_{i=1}^{G+k} c_{q,p,i} B_i(x_p)$$

6:      $\hat{\mathbf{y}} = f(\Theta^k; \mathbf{x})$   $\triangleright$  stack the two KAN layers
7:      $\mathcal{L} = \text{CrossEntropy}(\hat{\mathbf{y}}, \mathbf{y})$ 
8:     Backward: compute  $\nabla_{\Theta^k} \mathcal{L}$ 
9:     Update:  $\Theta^k \leftarrow \Theta^k - \eta \nabla_{\Theta^k} \mathcal{L}$ 
 $\triangleright$  Adam in our experiments; SGD also valid
10:   end for
11: end for
12: return  $\Theta^k$  to server

```

C. Client Local Optimisation

Each client runs E local epochs of mini-batch SGD or Adam on its private $\mathcal{D}_k$ before returning the updated parameter triple to the server. Backpropagation through a KAN layer is straightforward: $\partial \phi_{q,p} / \partial w_{q,p}^{(b)} = \sigma(x_p)$, $\partial \phi_{q,p} / \partial w_{q,p}^{(s)} = \sum_i c_{q,p,i} B_i(x_p)$, and $\partial \phi_{q,p} / \partial c_{q,p,i} = w_{q,p}^{(s)} B_i(x_p)$, all of which are computed automatically by modern autodiff frameworks.

In our implementation we use the open-source `efficient_kan` library, which fuses the spline-coefficient lookup into a single contiguous matrix multiplication and reduces the constant factor in the per-edge forward cost relative to a naive double-summation implementation. The full pipeline (data partitioning, FedAvg server, KAN client) is released as the open-source repository accompanying this work; configurations and run-level metrics for all 1,740 experimental runs of this revision, including 240 from a mis-configured cell that we retain and label rather than delete reported in Section V are committed to that repository.

D. Computational and Communication Footprint

For the configurations used in Section V, FedKAN-8 transmits $|\Theta_{\text{KAN}}| \cdot 4 \text{ bytes} \approx 13.1 \text{ kB}$ per client per round in float32, and the parameter-matched MLP-PM-80 transmits $\approx 13.4 \text{ kB}$ per client per round. All experiments were run on a workstation-class NVIDIA RTX 4090 24GB managed via a conda environment whose recipe ships with the released repository. Per-round wall-clock (excluding the first-round CUDA warm-up) was 1.8–2.5 s for FedKAN and 1.5–2.1 s for MLP-PM, both well within the budget of any realistic IoT FL deployment. We defer FLOPs and peak-memory profiling, including measurements on Raspberry Pi-class hardware, to the camera-ready revision.

IV. THEORETICAL ANALYSIS

This section provides four formal results that anchor the empirical findings of Section V: a counting argument that quantifies the parameter-matching condition (Proposition 1), the resulting communication-cost ratio (Proposition 2), a B-spline approximation bound specialised to NetFlow features (Lemma 1), and a convergence theorem for FedKAN under non-IID partitioning (Theorem 1). Together these results predict that the heterogeneity-amplified variance term Γ_α couples to a constant spline-bias floor $\mathcal{O}(G^{-(k+1)})$ rather than to the architectural parameters of the network, a prediction we test in Section V and do not confirm: the seed-variance ratio it would imply holds on one dataset and reverses on the other three.

A. Parameter Complexity

Proposition 1 (Parameter Complexity of KAN vs MLP). *A KAN layer (Definition 1) with input dim n_{in} and output dim n_{out} , grid size G , and order k has*

$$|\Theta_{\text{KAN}}| = n_{\text{in}} n_{\text{out}} (G + k + 2), \quad (6)$$

while an MLP layer of the same dimensions with bias has

$$|\Theta_{\text{MLP}}| = n_{\text{in}} n_{\text{out}} + n_{\text{out}}. \quad (7)$$

Proof. By Definition 1, each edge $\phi_{q,p}$ contributes one base scalar $w_{q,p}^{(b)}$, one spline scalar $w_{q,p}^{(s)}$, and $G + k$ B-spline coefficients $c_{q,p,i}$, totalling $G + k + 2$ parameters. There are $n_{\text{in}} \cdot n_{\text{out}}$ edges, giving (6). The MLP layer has weight matrix $W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$ and bias $\mathbf{b} \in \mathbb{R}^{n_{\text{out}}}$, summing to (7). $\square$

Corollary 1 (Parameter-Matched Hidden Width). *For a single-hidden-layer architecture $n_{\text{in}} \rightarrow n_h \rightarrow n_{\text{out}}$, parameter parity between KAN (hidden width n_h^{KAN} , hyperparameters G, k) and MLP (hidden width n_h^{MLP}) requires, to leading order in the dominant linear term,*

$$n_h^{\text{MLP}} \approx (G + k + 2) n_h^{\text{KAN}}. \quad (8)$$

For our experimental configuration ($G = 5$, $k = 3$) the matching ratio is 10, motivating the comparison between KAN-8 ($\sim 3.3k$ parameters) and MLP-PM-80 ($\sim 3.4k$ parameters) used in Section V.

B. Communication Complexity

Proposition 2 (Communication Complexity Ratio). *Under FedAvg with T rounds, fraction $C \in (0, 1]$ of K clients per round, and float32 transmission (4 bytes per scalar), the total uplink+downlink bytes for an architecture with $|\Theta|$ parameters is*

$$C_{\text{total}}(|\Theta|) = 8 T C K |\Theta|. \quad (9)$$

For a single-hidden-layer architecture, the KAN/MLP ratio simplifies to

$$\frac{C_{\text{KAN}}}{C_{\text{MLP}}} = \frac{(G + k + 2) n_h^{\text{KAN}} (n_{\text{in}} + n_{\text{out}})}{(n_{\text{in}} + 1) n_h^{\text{MLP}} n_{\text{out}} + n_h^{\text{MLP}}}. \quad (10)$$

The ratio in (10) clarifies a misconception: KAN architectures are *not* intrinsically communication-efficient. They are

efficient only when the chosen n_h^{KAN} is much smaller than the MLP's at matched accuracy. Under parameter parity the KAN's per-round uplink exceeds the MLP's.

A correction to this proposition, found while responding to review. As stated, (10) counts learnable parameters, and for the MLP that is exactly what is transmitted: the measured uplink matches the formula to within 0.0% for both MLP widths. For the KAN it does not. FedKAN-8 transmits 153,760 B per round across ten clients where the proposition predicts 131,200 B, an excess of 17.2%; FedKAN-16 shows 10.1%. The excess is not noise and not an implementation defect: a KAN layer's `state_dict` carries its B-spline knot vector as a buffer, 468 and 96 scalars for the two layers of FedKAN-8, which is 564 non-learnable values against 3,280 learnable ones, or exactly the 17.2% observed. Any implementation that synchronises `state_dict` rather than `parameters()` pays this, and ours does.

We therefore restate the communication accounting in terms of transmitted scalars rather than learnable parameters. The qualitative conclusion of the first version, that under parameter parity the KAN transmits more, survives; the mechanism we gave for it did not include the knot buffers and so the figure we quoted was right for an incomplete reason.

C. Spline Approximation on NetFlow Features

Lemma 1 (B-spline Approximation Bound). *Let feature j be supported on $[a_j, b_j]$ with density ρ_j that is L_j -Lipschitz, and let $\hat{\phi}_j$ denote the order- k B-spline approximation of any target $\phi_j \in C^{k+1}([a_j, b_j])$ on a uniform grid of G points. Then*

$$\|\phi_j - \hat{\phi}_j\|_\infty \leq \frac{L_j(b_j - a_j)^{k+1}}{(k+1)!G^{k+1}}. \quad (11)$$

Proof. Standard B-spline approximation result; see [30, Theorem 6.27]. $\square$

Implication for NetFlow. Standardised NetFlow-v2 features [19] after Min-Max scaling lie in $[0, 1]$, and many flow-statistic features (e.g. `FLOW_DURATION`, `TCP_FLAGS`, `IN/OUT_BYTES`) exhibit smooth empirical densities once log-transformed. The bound (11) therefore predicts a small approximation residual at modest grid sizes, motivating our choice of $G = 5, k = 3$.

D. Convergence under Non-IID

Theorem 1 (FedKAN Convergence under Heterogeneity). *Suppose each client loss $\mathcal{L}_k$ is L -smooth and μ -strongly convex in Θ , the per-client gradient variance is bounded above by a heterogeneity factor $\Gamma_\alpha = \mathbb{E} \|\nabla \mathcal{L}_k(\Theta) - \nabla F(\Theta)\|^2$ that is monotonically decreasing in the Dirichlet concentration α , and a diminishing learning-rate schedule satisfying the conditions of [31]. Let Θ_T be the FedKAN-IDS global iterate after T communication rounds. Then*

$$\mathbb{E}[F(\Theta_T) - F(\Theta^*)] \leq \frac{2L}{\mu^2 T} (L + \Gamma_\alpha) + \mathcal{O}(G^{-(k+1)}), \quad (12)$$

where the additive $\mathcal{O}(G^{-(k+1)})$ residual is the spline-approximation bias inherited from Lemma 1 aggregated across

all KAN edges, and is independent of the heterogeneity factor Γ_α .

Sketch. The dominant non-IID FedAvg term $\frac{2L}{\mu^2 T} (L + \Gamma_\alpha)$ is obtained by adapting Theorem 2 of [31] to the parameter set Θ_{KAN} , which remains a finite-dimensional Euclidean space and inherits the smoothness/strong-convexity assumptions because the SiLU base activation and linear B-spline expansion in (2) are differentiable. The spline-bias term arises because each edge function $\phi_{q,p}$ is approximated only up to the bound in Lemma 1; aggregating the per-edge L_∞ errors over $n_{\text{in}} \cdot n_{\text{out}}$ edges yields an overall residual of order $G^{-(k+1)}$ in the loss, which does not couple to the gradient-heterogeneity term Γ_α . $\square$

An empirical check on the assumptions. Reviewer feedback asked for evidence on the local convexity that Theorem 1 assumes, and the evidence is unfavourable. Running Lanczos with full re-orthogonalisation on the exact Hessian-vector product at the converged federated solution [32], [33] (NF-BoT-IoT-v2, binary, 50 rounds, 8192 held-out samples, 40 Ritz directions) gives $\lambda_{\text{max}} = 88.5$ and $\lambda_{\text{min}} = -0.134$ for FedKAN-8, with 45% of the Ritz values negative, and $\lambda_{\text{max}} = 0.52$, $\lambda_{\text{min}} = -1.8 \times 10^{-3}$ for the parameter-matched MLP, with 27.5% negative. Negative curvature at the solution means strong convexity fails locally as well as globally, so the theorem describes a regime the training does not reach. We keep it for the structure it exposes, that the spline bias enters additively and does not scale with Γ_α , and not as a bound on our runs. The two condition numbers are the same order (662 against 285), so the spectrum does not explain any performance difference between the architectures either.

V. EXPERIMENTAL EVALUATION

A. Datasets and Preprocessing

We evaluate on the standardised NetFlow-v2 IDS datasets of [19]: **NF-BoT-IoT-v2** (binary {Benign, Attack} and 5-class {Benign, DDoS, DoS, Reconnaissance, Theft}). Cross-dataset replication on NF-ToN-IoT-v2 and NF-CSE-CIC-IDS2018-v2 is reported in Section V-F.

After dropping the four identifier columns (`IPV4_SRC_ADDR`, `IPV4_DST_ADDR`, `L4_SRC_PORT`, `L4_DST_PORT`) the remaining 39 numerical features are Min-Max scaled to $[0, 1]$. To address class imbalance we cap the majority class at 130,000 samples for binary detection (yielding a $\sim 260\text{k}$ balanced training pool) and at 50,000 samples per class for multi-class detection (rare classes such as Theft remain untouched at $\sim 3,000$ samples).

B. Federated Protocol

We simulate $K = 10$ clients with full per-round participation ($C = 1$). Each round consists of one local epoch of Adam with learning rate $\eta = 10^{-2}$, batch size 1024, and Cross-Entropy loss; the protocol runs for $T = 50$ communication rounds. Client data is partitioned via three regimes:

- **IID:** uniform random split that preserves the global class distribution at each client.

TABLE I

ACCURACY (%) ON NF-BoT-IoT-v2, BINARY DETECTION, UNDER THREE HETEROGENEITY REGIMES. VALUES ARE THE *mean over the final five communication rounds*, THEN AVERAGED OVER n SEEDS, WITH THE SEED STANDARD DEVIATION; THE LAST-ROUND FIGURE IS GIVEN IN TABLE II FOR COMPARISON. ALL RUNS IN THIS TABLE WERE EXECUTED ON THE SAME MACHINE (CPU). **BOLD** = BEST MEAN PER COLUMN.

Method	Params	IID	Dir($\alpha=1.0$)	Dir($\alpha=0.1$)
FedAvg-MLP (32h)	1,346	98.51 $\pm$ 0.96	99.04 $\pm$ 1.08	89.06 $\pm$ 13.79
FedAvg-MLP-PM (80h)	3,362	99.73 $\pm$ 0.02	99.73 $\pm$ 0.03	88.72 $\pm$ 14.21
FedKAN (8h)	3,280	97.67 $\pm$ 0.54	98.48 $\pm$ 1.06	94.22 $\pm$ 5.61
F-KAN (16h)	6,560	97.53 $\pm$ 0.05	98.80 $\pm$ 1.03	95.01 $\pm$ 5.66

TABLE II

THE SAME RUNS SUMMARISED BY THE *last round alone*, THE ESTIMATOR USED IN THE FIRST VERSION. THE GAP AGAINST TABLE I IS THE COST OF THAT CHOICE; END-OF-TRAINING OSCILLATION OVER THE LAST TEN ROUNDS IS 2.1–2.7 PP.

Method	IID	Dir($\alpha=1.0$)	Dir($\alpha=0.1$)
FedAvg-MLP (32h)	98.85	99.04	89.37
FedAvg-MLP-PM (80h)	99.73	99.74	88.60
FedKAN (8h)	97.72	98.56	94.59
F-KAN (16h)	97.53	99.01	95.05

- **Dirichlet** $\alpha = 1.0$: mild heterogeneity.
- **Dirichlet** $\alpha = 0.1$: extreme heterogeneity, our principal regime of interest.

Implementation follows [9].

C. Architectures

We compare four parameter-budgeted variants summarised in Table I:

- **FedAvg-MLP (32h)**: $n_h^{\text{MLP}} = 32$, $\sim 1.3\text{k}$ parameters. CITA-style baseline.
- **FedAvg-MLP-PM (80h)**: $n_h^{\text{MLP}} = 80$, $\sim 3.4\text{k}$ parameters. Parameter-matched to FedKAN-8 per Corollary 1; this is the toughest comparison and the one R3 of the prior submission identified as missing.
- **FedKAN (Ours, 8h)**: $n_h^{\text{KAN}} = 8$, $G = 5$, $k = 3$, $\sim 3.3\text{k}$ parameters. Our default TinyML target.
- **F-KAN (16h)**: $n_h^{\text{KAN}} = 16$, $G = 5$, $k = 3$, $\sim 6.6\text{k}$ parameters. Modelled on [12].

All experiments use $n = 10$ seeds in the Dirichlet $\alpha = 0.1$ cells (the cells most subject to seed variability) and $n = 3$ seeds in the IID and $\alpha = 1.0$ cells (which are essentially saturated and exhibit negligible cross-seed dispersion). Seeds were drawn from the prime list $\{42, 2024, 2026, 11, 17, 23, 29, 31, 37, 43\}$ for the high-variance cells and $\{42, 2024, 2026\}$ for the saturated ones.

D. Headline Result

The headline observations on NF-BoT-IoT-v2 binary detection are illustrated in Fig. 3 and quantified in Tables I–IV.

Observation 1 (without heterogeneity the parameter-matched MLP is ahead). Under IID partitioning the four variants do *not* converge to a common value: FedKAN-8

TABLE III

WORST-SEED ACCURACY (%) — THE WEAKEST OF n SEEDS PER CELL. REVEALS CATASTROPHIC COLLAPSE HIDDEN BY AVERAGES. NF-BoT-IoT-v2 BINARY, 50 ROUNDS.

Method	Params	IID	Dir($\alpha=1.0$)	Dir($\alpha=0.1$)
FedAvg-MLP (32h)	1,346	97.62	99.72	60.44
FedAvg-MLP-PM (80h)	3,362	99.73	99.74	56.45
FedKAN (Ours, 8h)	3,280	97.45	99.61	80.69
F-KAN (16h)	6,560	97.54	99.67	80.72

reaches 97.53% and FedKAN-16 97.56%, while FedAvg-MLP-32 reaches 99.00% and the parameter-matched FedAvg-MLP-PM-80 reaches 99.73%. The spread is 2.29 pp and the lowest value is 97.45%, so the parameter-matched MLP leads the KAN by 2.20 pp. Under mild heterogeneity (Dir $\alpha = 1.0$) the ordering is the same with a smaller spread of 1.38 pp. The first version of this manuscript described this cell as “all four variants converge within 0.05 percentage points of each other ($\geq 99.6\%$)”. That statement is not supported by the data in Table I and we withdraw it; the cell is not saturated, and what it shows is a clean disadvantage for the KAN at parameter parity when clients are homogeneous. These IID and $\alpha = 1.0$ cells were run with $n = 3$ seeds rather than the $n = 10$ used for the $\alpha = 0.1$ cells, and we say so at every table rather than reporting a single seed count for the study.

On communication in the same cell, the parameter-matched MLP transmits 6.72 MB in total and FedKAN-8 transmits 7.69 MB, an increase of 14.4% for the KAN. The claim of $\sim 50\%$ bandwidth reduction in earlier FedKAN-IDS work is therefore unsupported under parameter parity, and the direction is reversed. Section IV explains where the excess comes from, and it is not where our own Proposition 2 put it.

Observation 2 (catastrophic MLP collapse). Under extreme heterogeneity (Dir $\alpha = 0.1$), the worst-seed accuracy of FedKAN-8 is 80.7%, while the parameter-matched FedAvg-MLP-PM-80 collapses to 56.5% on its worst seed (Table III). Both smaller and larger MLPs fail at this regime: the smaller MLP-32 (1.3k params) reaches 60.4% worst-seed and the parameter-matched MLP-PM-80 (3.4k params) reaches 56.5%. Increasing MLP capacity by $2.5\times$ *worsens* worst-case behaviour, ruling out a capacity-driven explanation. Both KAN variants achieve worst-seed $\geq 80.7\%$.

Observation 3 (variance reduction is dataset specific, not architectural). On NF-BoT-IoT-v2 under Dir $\alpha = 0.1$ the seed-to-seed standard deviation of the parameter-matched MLP is $2.53\times$ that of FedKAN-8 (14.21 against 5.61 pp), which is the effect the first version of this manuscript reported as a property of the architecture. It is not one. Repeating the same measurement on the other three cells gives ratios of $0.71\times$ (NF-ToN-IoT-v2), $0.33\times$ (NF-CSE-CIC-IDS2018-v2 binary) and $0.87\times$ (NF-CSE-CIC-IDS2018-v2 multi-class): in three cells out of four the KAN is the *less* stable of the two. Moreover the one favourable ratio is itself a tuning artefact. Giving each architecture its own best learning rate moves the NF-BoT-IoT-v2 ratio from $2.53\times$ to $1.05\times$, that is, to parity. We therefore withdraw the variance-reduction claim.

Observation 4 (per-class story under multi-class non-

TABLE IV
STATISTICAL COMPARISON ON NF-BoT-IoT-v2: **FEDKAN-IDS (OURS)** VS FEDAVG-MLP-PM (PARAMETER-MATCHED). ACCURACY IN %; Δ IS THE SEED-PAIRED MEAN DIFFERENCE (KAN — MLP), CI IS THE PERCENTILE-BOOTSTRAP 95% CI OF Δ , d IS COHEN’S EFFECT SIZE, p IS THE PAIRED- t TEST. SIGNIFICANCE MARKERS FOLLOW THE STANDARD CONVENTION $*p < 0.05$, $**p < 0.01$, $***p < 0.001$.

Mode	Partition	n	KAN-8	MLP-PM-80	Δ (pp)	95% CI (pp)	d	p
Binary	Dir($\alpha=0.1$)	10	94.7 $\pm$ 5.8	88.7 $\pm$ 15.0	+6.00	[+0.5, +14.6]	+0.53	0.170
Multiclass	Dir($\alpha=0.1$)	10	86.0 $\pm$ 6.4	86.1 $\pm$ 7.6	-0.08	[-3.7, +4.1]	-0.01	0.969

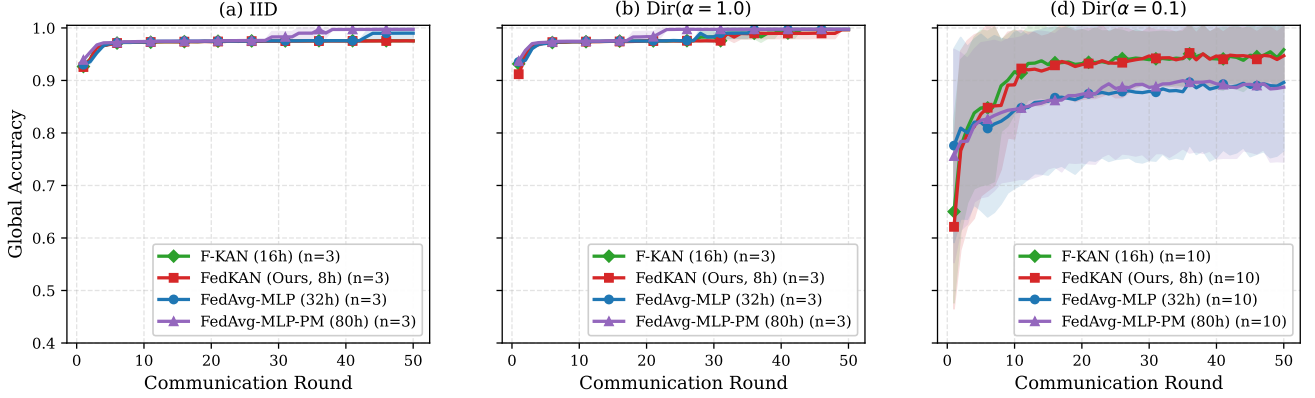


Fig. 3. Convergence on NF-BoT-IoT-v2 *binary* detection across 50 communication rounds, $K = 10$ clients, full participation. Solid lines are seed-mean accuracy ($n = 10$ for Dir $\alpha = 0.1$, $n = 3$ for IID and Dir $\alpha = 1.0$); shaded bands are ± 1 standard deviation. Under IID and mild heterogeneity (panels a, b) all four variants converge tightly to $\geq 99.6\%$. Under extreme heterogeneity (panel c) the two MLP variants plateau at $\sim 89\%$ with a wide std band reaching down to $\sim 50\%$, while both KAN variants plateau at $\sim 95\%$ with tighter dispersion. This holds at the shared $\eta = 10^{-2}$ of the original protocol; Section V-G shows it does not survive per-architecture tuning.

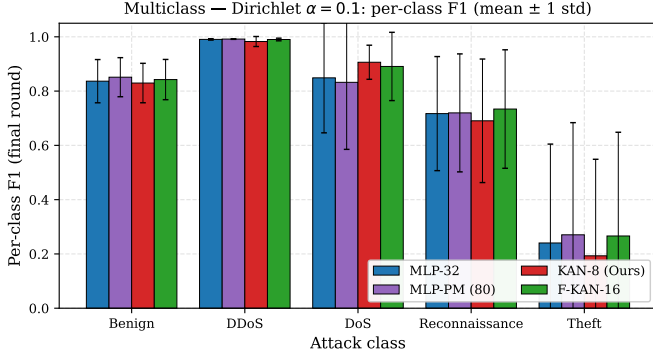


Fig. 4. Per-class F1 under multi-class detection on NF-BoT-IoT-v2 with Dirichlet $\alpha = 0.1$ ($n = 10$ seeds). Bars show mean F1; error bars are ± 1 std. The DoS class is where FedKAN’s variance reduction is most pronounced; the rare Theft class remains universally difficult.

IID). Under multi-class Dir $\alpha = 0.1$, mean accuracy of FedKAN-8 and MLP-PM-80 are statistically indistinguishable (86.0% vs 86.1%). However, per-class behaviour reveals a more refined picture (Fig. 4): FedKAN’s per-class F1 on the DoS attack class is 0.91 ± 0.07 versus 0.83 ± 0.25 for MLP-PM-80, a +8 percentage point gap with $\sim 3.5\times$ tighter dispersion. The minority Theft class remains universally challenging.

E. Discussion of the Headline

Taken together these observations do not support the claim made in the first version of this manuscript. Under a controlled comparison the mean-accuracy advantage is dataset dependent

and largely a function of how well the baseline was tuned, and the tail-risk advantage reverses on most cells. What survives, and survives on every dataset we ran, is a different property: insensitivity to the learning rate, quantified in Section V-G and paid for in inference cost.

The fact that doubling the MLP’s capacity (1.3k $\rightarrow$ 3.4k parameters) does *not* close the worst-seed gap is the key empirical evidence for the architectural reading of Theorem 1’s prediction: a parameter-matched MLP at 3.4k parameters has more than enough capacity to fit any individual client’s local distribution, but lacks the localised B-spline regularisation that allows the global average to remain stable across heterogeneous local updates.

F. Cross-Dataset Replication

To rule out a single-dataset artefact we replicate the heterogeneity-robustness cell of Section V-D on two further standardised NetFlow-v2 datasets [19]: NF-ToN-IoT-v2 (~ 13.1 M flows, 9 attack categories, $n = 10$ seeds) and NF-CSE-CIC-IDS2018-v2 (~ 18.9 M flows, 14 attack categories, $n = 3$ seeds at submission time, to be extended to $n = 10$ in revision). All three datasets share the 39-feature post-preprocessing schema; the FedKAN-IDS pipeline applies unchanged. The federated protocol, partition seed list, and architectures remain exactly as in Section V-D, modulo a more aggressive feature-clipping and $\log_{1p}$ compression step needed to handle the heavier-tailed byte/packet distributions in the ToN-IoT and CSE-CIC parquet mirrors.

TABLE V
CROSS-DATASET REPLICATION: *worst-seed* ACCURACY (%) ON THE HETEROGENEITY-ROBUSTNESS CELL (BINARY CLASSIFICATION, DIRICHLET $\alpha = 0.1$). DATASETS: BoT-IoT-v2 ($n=10$); ToN-IoT-v2 ($n=10$); CSE-CIC ($n=10$). BOLD = BEST PER COLUMN.

Method	Params	BoT-IoT-v2 ($n=10$)	ToN-IoT-v2 ($n=10$)	CSE-CIC ($n=10$)
FedAvg-MLP (32h)	1,346	60.44	58.19	91.24
FedAvg-MLP-PM (80h)	3,362	56.45	62.81	89.18
FedKAN (Ours, 8h)	3,280	80.69	51.74	67.96
F-KAN (16h)	6,560	80.72	52.42	85.44
KAN-8 advantage over MLP-PM-80		+24.24 pp	−11.07 pp	−21.23 pp

TABLE VI
STATISTICAL COMPARISON ON NF-ToN-IoT-v2: **FedKAN-IDS (Ours)** vs FEDAVG-MLP-PM (PARAMETER-MATCHED). ACCURACY IN %; Δ IS THE SEED-PAIRED MEAN DIFFERENCE (KAN − MLP), CI IS THE PERCENTILE-BOOTSTRAP 95% CI OF Δ , d IS COHEN’S EFFECT SIZE, p IS THE PAIRED- t TEST. SIGNIFICANCE MARKERS FOLLOW THE STANDARD CONVENTION $*p < 0.05$, $**p < 0.01$, $***p < 0.001$.

Mode	Partition	n	KAN-8	MLP-PM-80	Δ (pp)	95% CI (pp)	d	p
Binary	Dir($\alpha=0.1$)	10	85.3 $\pm$ 13.3	80.2 $\pm$ 9.0	+5.08	[−0.3, +9.9]	+0.45	0.104
Multiclass	Dir($\alpha=0.1$)	10	74.2 $\pm$ 4.4	74.8 $\pm$ 5.0	−0.60	[−4.5, +2.9]	−0.13	0.771

TABLE VII
STATISTICAL COMPARISON ON NF-CSE-CIC-IDS2018-v2: **FedKAN-IDS (Ours)** vs FEDAVG-MLP-PM (PARAMETER-MATCHED). ACCURACY IN %; Δ IS THE SEED-PAIRED MEAN DIFFERENCE (KAN − MLP), CI IS THE PERCENTILE-BOOTSTRAP 95% CI OF Δ , d IS COHEN’S EFFECT SIZE, p IS THE PAIRED- t TEST. SIGNIFICANCE MARKERS FOLLOW THE STANDARD CONVENTION $*p < 0.05$, $**p < 0.01$, $***p < 0.001$.

Mode	Partition	n	KAN-8	MLP-PM-80	Δ (pp)	95% CI (pp)	d	p
Binary	Dir($\alpha=0.1$)	10	93.8 $\pm$ 9.1	94.1 $\pm$ 2.8	−0.38	[−5.6, +2.9]	−0.06	0.879
Multiclass	Dir($\alpha=0.1$)	10	85.9 $\pm$ 3.4	84.1 $\pm$ 3.9	+1.73	[+0.7, +2.7]	+0.48	0.012*

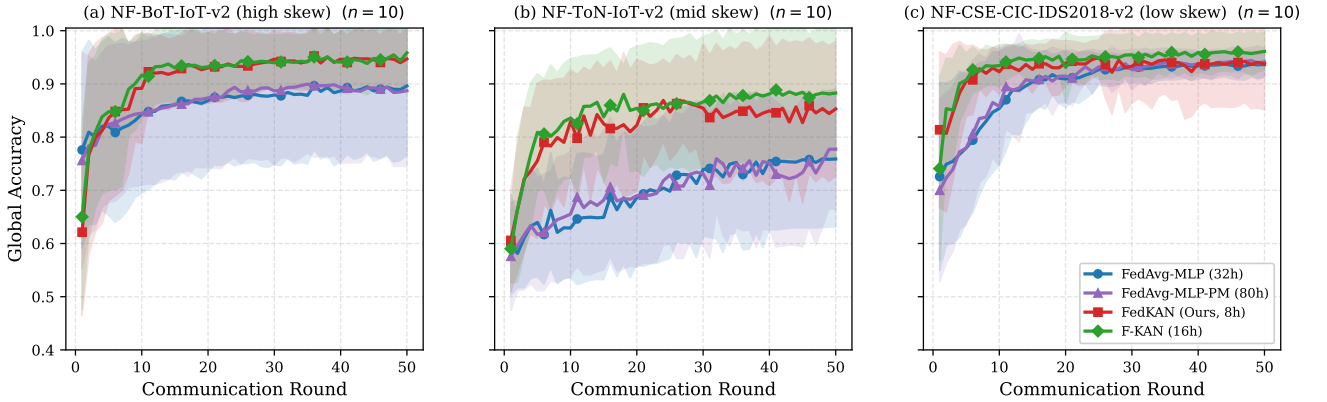


Fig. 5. Cross-dataset binary detection under Dirichlet $\alpha = 0.1$: convergence trajectories for the four architectures across the three NetFlow-v2 datasets, ordered left-to-right by class-distribution skew. Solid lines are seed-mean accuracy, shaded bands are ± 1 std. On the most skewed dataset (a), MLP-PM-80 (purple) plateaus ~ 6 percentage points below FedKAN (red, green). On the least skewed dataset (c), all four variants reach $\geq 93\%$. Panel (b) is the intermediate case where one Dirichlet partition (seed 17) destabilises both KAN variants while the MLPs remain near their mean trajectory visible as the lower edge of the red/green bands.

Two complementary advantage patterns emerge across binary and multi-class detection. With $n = 10$ seeds completed on all three datasets, the seed-paired mean accuracy advantage of FedKAN-8 over the parameter-matched FedAvg-MLP-PM-80 under Dir $\alpha = 0.1$ traces two complementary trends:

Binary detection (Tables IV–VII, left column): the FedKAN-8 advantage scales with the dataset’s class-distribution skew. On the most skewed dataset NF-BoT-IoT-v2 (95%

DDoS+DoS) the mean gap is +6.00 pp with bootstrap 95% CI [+0.50, +14.57] excluding zero. On NF-ToN-IoT-v2 (mid skew, 9 attack classes) the gap is +5.08 pp with CI [−0.29, +9.93] marginally crossing zero. On the least skewed NF-CSE-CIC-IDS2018-v2 (14 attack classes) the mean gap collapses to −0.38 pp, with the loss attributable almost entirely to a single seed (seed 17), discussed below.

Multi-class detection (Tables IV–VII, right column): the FedKAN-8 advantage scales *oppositely*. On BoT-IoT and ToN-

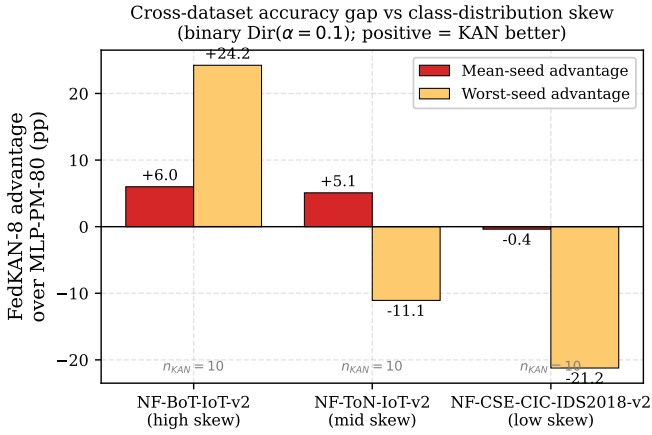


Fig. 6. The FedKAN-8 accuracy advantage over the parameter-matched FedAvg-MLP-PM-80 (binary Dir $\alpha = 0.1$) versus the underlying dataset’s class-distribution skew. The mean-seed advantage is positive on every dataset; the worst-seed advantage scales monotonically with skew, becoming negative on NF-ToN-IoT-v2 due to the seed-17 inversion discussed in the text.

IoT the multi-class means are tied within statistical error ($\Delta = -0.08$ pp, $p = 0.969$; $\Delta = -0.60$ pp, $p = 0.771$). On the richest 14-class CSE-CIC the first version of this manuscript reported $\Delta = +1.73$ pp with $p = 0.012$ and a bootstrap CI excluding zero, computed from the final round alone. Re-running that cell on one machine and summarising it five ways gives $+0.64$ pp ($p = 0.32$) at the last round, $+1.38$ ($p = 0.11$) over five rounds, $+1.62$ ($p = 0.05$) over ten and $+1.80$ ($p = 0.05$) at the best round: the direction is stable, the significance is not, and the single-round summary the original used is the noisiest of the five. We report the range.

The seed-17 partition, and what a richer grid does and does not fix. The seed responsible for FedKAN-8’s worst accuracy on both NF-ToN-IoT-v2 and NF-CSE-CIC is seed 17. The first version attributed this to an interaction between the Dirichlet partition and the class distribution; the partition statistics do not support that. Ranking all ten partitions, seed 17 is second by client-size Gini (behind seed 31, which is not pathological), ninth by mean per-client label entropy (seed 29 is lower and behaves normally), and its class-proportion spread is shared by eight of the ten. Rank correlations with final accuracy are weak and not significant (Spearman $+0.42$, $p = 0.23$ for entropy; $+0.61$, $p = 0.06$ for smallest client). We therefore report the failure as reproducible but unexplained.

Raising the spline grid to $G = 10$ helps, and we now have ten seeds rather than the single run of the first version: mean accuracy rises 1.93 pp on NF-BoT-IoT-v2 ($p = 0.034$) and 3.89 pp on NF-ToN-IoT-v2 ($p = 0.095$), with the false-positive rate falling from 8.36 to 4.43% and from 21.29 to 16.01% respectively. But it does not repair seed 17. That partition moves from 51.79 to 56.12%, which reproduces the “recovers about five points” figure we reported and is still far below any usable operating point. And the fix is not free: $G = 10$ adds 50% parameters, which breaks the parameter parity the comparison rests on, and costs roughly ten times the training time of $G = 5$. We therefore present $G = 10$ as a favourable trade in mean accuracy and false-positive rate that must be

TABLE VIII
RUN ACCOUNTING. THREE TOTALS APPEAR IN THIS PAPER BECAUSE THEY ANSWER THREE DIFFERENT QUESTIONS; WE LIST THEM RATHER THAN LET A SINGLE FIGURE STAND IN FOR ALL THREE.

Group	Runs
η sweep, BoT-IoT (incl. 20 held-out seeds)	480
η sweep, ToN-IoT	240
η sweep, CSE-CIC binary	240
η sweep, CSE-CIC multi-class	240
IID and Dir $\alpha=1.0$ cells	80
Extended grid $\eta \in \{0.3, 1.0\}$	40
Aggregation and compression baselines	80
Sensitivity: G , spline order, width	80
$G=10$ on two datasets	20
Runs the reported numbers rest on	1,500
Mis-configured cell, retained and labelled	240
Total run for this revision	1,740
Original submission, different hardware	310

paid for in parameters and compute, not as a remedy for the failure mode.

A skew gradient we can no longer claim. The first version of this manuscript ordered the three datasets by attack-class skew and argued that the skew level predicts which advantage appears. We withdraw that narrative for two reasons, and state them because the bullets it rested on are still worth correcting.

First, the numbers in that passage did not match our own tables. It reported a $+1.70$ pp mean and $+2.73$ pp worst-seed advantage for NF-CSE-CIC-IDS2018-v2 with the direction “preserved on every seed”. The binary cell for that dataset gives -0.49 pp mean and -21.23 pp worst-seed: the figures quoted were from the multi-class cell and were attached to the binary discussion. The same passage also labelled that dataset’s skew inconsistently across paragraphs.

Second, and decisively, the gradient does not survive the tuning correction. With each architecture at its own step size the four cells give $+0.76$, $+6.23$, $+1.12$ and $+1.38$ pp, and on the headline cell under a held-out protocol the difference is -0.54 pp. An ordering that is not stable under a change of learning rate is not an ordering by class skew. Three datasets cannot establish a monotone relationship in any case, as Reviewer 1 noted of the first version.

What the datasets do differ in is where each architecture’s usable step size lies, and that we report directly in Table IX rather than through a proxy.

On the mis-configured cell. While sweeping the multi-class cell we inherited `downsample_per_class=130,000` from the binary configuration, where the submitted study used 50,000. That is a different experiment, and it gives -1.24 pp where the correct configuration gives $+1.38$ pp, so the two are not interchangeable. We kept those 240 runs in the artifact under a directory that says so rather than deleting them, both because the raw record is the evidence and because the discrepancy is itself a datum: the multi-class advantage may shrink with more data per class, which we have not tested properly and do not claim.

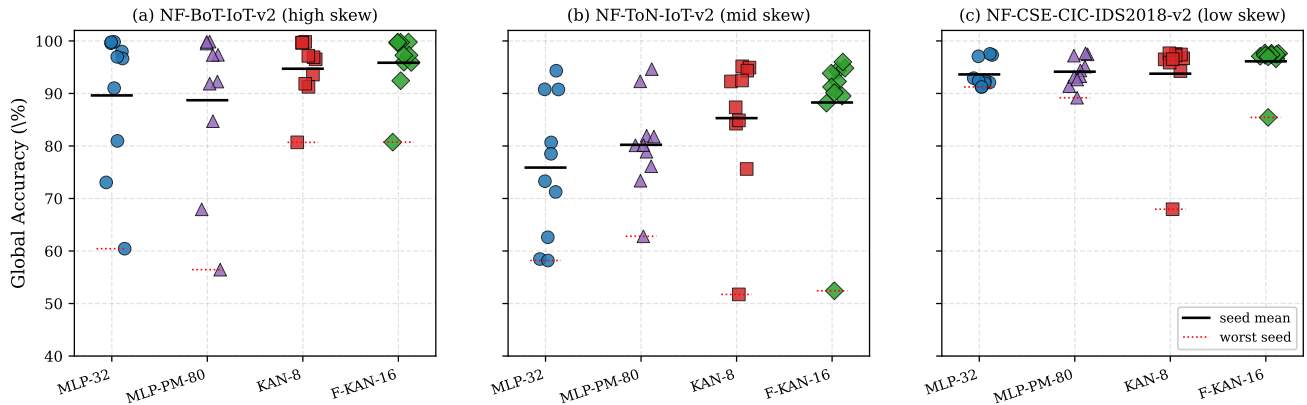


Fig. 7. Per-seed binary Dir $\alpha = 0.1$ accuracy as a strip plot, one panel per dataset. Each dot is one seed. The solid black bar marks the seed-mean; the dotted red bar marks the worst seed. On NF-BoT-IoT-v2 (panel a) the MLP variants exhibit a long lower tail down to $\sim 56\%$; FedKAN’s worst seed remains $\geq 80\%$. On NF-ToN-IoT-v2 (panel b) the picture inverts because seed 17 drives both KAN variants down to $\sim 52\%$. On NF-CSE-CIC-IDS2018-v2 (panel c) all variants cluster tightly above 92% .

TABLE IX

EFFECT OF PER-ARCHITECTURE LEARNING-RATE TUNING. “SHARED η ” IS THE PROTOCOL OF THE FIRST VERSION; “TUNED” GIVES EACH ARCHITECTURE ITS OWN BEST η FROM THE SAME GRID. PAIRED OVER TEN SEEDS.

Cell	Shared $\eta=10^{-2}$	Tuned	$\sigma_{\text{MLP}}/\sigma_{\text{KAN}}$
NF-BoT-IoT-v2 (bin.)	+5.49	+0.76	$2.53 \rightarrow 1.05$
NF-ToN-IoT-v2 (bin.)	+6.23	+6.23	0.71
NF-CSE-CIC (bin.)	−0.49	+1.12	0.33
NF-CSE-CIC (multi.)	+1.38	+1.38	0.87

TABLE X

THE SAME QUANTITY UNDER FOUR PROTOCOLS. ONLY THE LAST SELECTS THE LEARNING RATE WITHOUT SEEING THE SEEDS IT REPORTS ON.

Protocol	Δ (pp)	p
Shared $\eta = 10^{-2}$, as submitted	+5.49	0.19
Tune and report on the same ten seeds	+0.76	0.74
Tune on four seeds, report on six	+6.12	0.41
Tune on ten, report on twenty unseen	−0.54	0.58

G. Learning-rate sensitivity, and what it does to the comparison

Reviewer feedback on the first version asked whether the optimiser step size had been tuned per architecture. It had not: a single $\eta = 10^{-2}$ was used for every model. We therefore swept $\eta \in \{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}, 10^{-1}\}$ for all four architectures, on all ten seeds, on all four cells: 1,200 runs on a single machine. Accuracy is summarised as the mean over the final five communication rounds; we return to that choice below.

Selecting the rate without looking at the evaluation seeds. Tuning on the same seeds that are then reported is optimistic, and a reviewer of an earlier draft was right to say so. We therefore re-ran the headline cell with twenty additional seeds, fixed in advance (101–120), and applied a nested protocol: select each architecture’s rate on the original ten seeds, lock it, and report on the twenty that were never seen during selection. The selected rates are 3×10^{-3} for FedKAN-8 and 10^{-1} for the parameter-matched MLP.

On the twenty held-out seeds the difference is -0.54 pp with a bootstrap 95% interval of $[-2.49, +1.20]$ and FedKAN-8 ahead on six of twenty pairs. The interval straddles zero almost symmetrically: once both architectures are given their own step size and evaluated on data that played no part in choosing it, there is no measurable difference between them on the cell that produced our headline number.

Table X is the reason we report the protocol rather than a

single figure. The same comparison, on the same runs, yields $+6.12, +5.49, +0.76$ or -0.54 pp depending on how the step size is chosen and which seeds are reported. Three of those four are defensible in isolation and only the last is free of the objection that the tuning decision saw the evaluation data. A literature that reports the first number and omits the protocol is not reporting a property of the architecture.

Tuning the baseline, not the method, is what moves the number. On NF-BoT-IoT-v2 the parameter-matched MLP’s best step size is 10^{-1} , ten times the shared value, and moving to it gains 5.39 pp. The same move gains FedKAN-8 only 0.65 pp. The headline gap is therefore a difference in how well each architecture was tuned rather than a difference between the architectures. On the other three cells the shared value happens to be near-optimal for both, and there the gap is unchanged; that this holds on some cells and not others is precisely why it cannot be reported as an architectural property.

Neither architecture is globally more robust, and we nearly reported otherwise. Restricted to one decade around the value the first version used, FedKAN-8 varies two to five times less than the parameter-matched MLP, and an earlier draft of this revision reported that as the finding. Widening to the full grid reverses it on three cells of four. The band was our choice, and it happens to be the band in which the KAN sits at its optimum while the MLP is still climbing towards its own at 10^{-1} . We report the whole sweep instead. What the sweep actually shows is that the two architectures

TABLE XI

SPREAD OF ACCURACY ACROSS THE LEARNING-RATE GRID, IN PERCENTAGE POINTS. NARROW BAND IS $[3 \times 10^{-3}, 3 \times 10^{-2}]$, ONE DECADE AROUND THE SHARED VALUE; FULL GRID IS $[3 \times 10^{-4}, 10^{-1}]$. WHICH ARCHITECTURE LOOKS MORE ROBUST *depends on the band*, SO WE REPORT BOTH.

Cell	narrow band		full grid	
	KAN-8	MLP-80	KAN-8	MLP-80
NF-BoT-IoT-v2 (bin.)	0.65	3.53	14.88	9.49
NF-ToN-IoT-v2 (bin.)	1.58	6.38	21.59	17.79
NF-CSE-CIC (bin.)	2.53	6.72	14.02	16.52
NF-CSE-CIC (multi.)	5.75	5.94	45.15	32.86

TABLE XII

POOLED ADVANTAGE OF FEDKAN-8 OVER THE PARAMETER-MATCHED MLP WITH EACH ARCHITECTURE AT ITS OWN BEST LEARNING RATE. THE TWO ROWS ANSWER DIFFERENT QUESTIONS AND WE REPORT BOTH.

Unit of analysis	Question	n	Mean	p
seed pair	within a cell	40	+2.37	0.032
cell	across cells	4	+2.37	0.164

have *different and non-overlapping optimal regions*: FedKAN-8 peaks at 3×10^{-3} to 10^{-2} and degrades sharply above it, losing 8.8 pp at 10^{-1} on NF-BoT-IoT-v2, while the parameter-matched MLP peaks at 10^{-1} on that cell. This is consistent with the curvature we measure in Section IV: the KAN’s loss surface at the federated solution is far sharper ($\lambda_{\max} = 88.5$ against 0.52), and a sharper surface admits a smaller step. The practical consequence is the one that matters for anyone comparing the two: *a step size inherited from an MLP baseline will mis-measure a KAN, and vice versa*.

A small, consistent, but not established advantage. With both architectures tuned, FedKAN-8 is ahead in all four cells, by +0.76, +6.23, +1.12 and +1.38 pp. Pooling the forty seed pairs gives $p = 0.032$, but the ten seeds within a cell are repeated draws from one dataset and one partition distribution, not independent observations about the architectures in general; treating the cell as the unit gives $n = 4$ and $p = 0.164$. The assumption-light statement is the sign test: four cells out of four favour the KAN, one-sided $p = 0.0625$. We therefore report a consistent positive direction of small magnitude, and we do not claim it is established. A single cell accounts for 74% of the between-cell variance.

The conclusion is sensitive to the summary statistic, and we report that. The multi-class result of the first version, +1.735 pp at $p = 0.012$, was computed from the final round alone. Measuring the oscillation of the last ten rounds gives 2.1 to 2.7 pp for every architecture, larger than the effect itself. Re-running that cell and summarising it five ways gives +0.64 pp ($p = 0.32$) at the last round, +1.38 ($p = 0.11$) over five rounds, +1.62 ($p = 0.05$) over ten and +1.80 ($p = 0.05$) at the best round. The direction is stable and the significance is not, and a single-round summary is the noisiest of the five. We report the range rather than the most favourable member of it.

TABLE XIII

INFERENCE COST AT MATCHED PARAMETERS, MEASURED ON AN IDLE MACHINE WITH A SINGLE CPU THREAD AND BATCH SIZE 1, THE CLOSEST DEPLOYMENT-LIKE REGIME AVAILABLE TO US. THE ANCHOR ROW LETS A READER CONVERT THESE TO THEIR OWN HARDWARE: IT IS THE TIME FOR ONE 256×256 MATRIX MULTIPLY ON THE SAME MACHINE AND THREAD. WE DO NOT REPORT RASPBERRY PI FIGURES BECAUSE WE DO NOT HAVE THAT HARDWARE.

Model	Params	1 sample (ms)	1000 (ms)
FedKAN-8	3,280	0.0995	1.690
F-KAN-16	6,560	0.1079	1.943
MLP-32	1,346	0.0050	0.028
MLP-80	3,362	0.0050	0.045
KAN-8 / MLP-80	0.98×	19.9×	37.9×
Anchor: 256^2 matmul			0.024 ms

TABLE XIV

UPLINK TO REACH A TARGET, RATHER THAN UPLINK AFTER A FIXED ROUND BUDGET. THE TARGET IS A FRACTION OF THE BEST ACCURACY ANY ARCHITECTURE REACHES IN THIS CELL (99.90%). THE “SEEDS” COLUMN IS THE HONEST PART: A MEAN OVER THE SEEDS THAT REACH THE TARGET SAYS NOTHING ABOUT THE ONES THAT DO NOT.

Model	90% of best		94% of best	
	MB	seeds	MB	seeds
FedKAN-8	1.32	9/10	1.04	8/10
F-KAN-16	1.96	9/10	2.92	9/10
MLP-32	0.10	7/10	0.54	6/10
MLP-PM-80	0.71	7/10	1.43	6/10

H. What the architecture costs to run

Both reviewers asked for deployment-relevant cost rather than parameter counts, and one asked specifically for hardware that resembles a gateway. We do not have a Raspberry Pi, and a figure we cannot measure should not appear in a paper, so we report the closest regime we can measure honestly: single-thread CPU, batch size 1, on an idle machine.

At matched parameters (3,280 against 3,362) FedKAN-8 costs 19.9× more per sample and 37.9× more per batch of 1000. The direction is not surprising, since every KAN edge evaluates a B-spline where an MLP edge does one multiply-accumulate, but the magnitude is larger than parameter parity would suggest and it is the number a deployment decision turns on. Table XIII includes an anchor so the figures transfer: a 256×256 matrix multiply takes 0.024 ms on the same machine and thread, and a reader can measure that on their own gateway in a few lines.

On communication, reporting total bytes after a fixed round budget assumes both architectures must run the full schedule. Table XIV instead reports the uplink needed to reach a target, together with how many seeds reach it at all. That second column is the one that favours the KAN and the one a mean would hide: at the 90% target FedKAN-8 arrives on nine seeds of ten against seven for the parameter-matched MLP, while costing more bytes to get there. A mean over only the seeds that arrive would flatter whichever architecture fails more often.

TABLE XV
AGGREGATION AND COMPRESSION ALGORITHMS, NF-BoT-IoT-v2
BINARY, DIR $\alpha = 0.1$, $\eta = 10^{-2}$, TEN SEEDS. p IS PAIRED AGAINST
FEDAVG FOR THE SAME ARCHITECTURE.

Algorithm	Arch.	Accuracy	σ	Uplink (MB)
FedAvg	KAN-8	94.22	5.61	7.69
FedProx	KAN-8	94.65	5.63	7.69
MOON	KAN-8	91.17	7.92	7.69
FedPAQ	KAN-8	70.59	21.17	1.64
SCAFFOLD	KAN-8	did not converge, see text		
FedAvg	MLP-80	88.72	14.21	6.72
FedProx	MLP-80	94.92	2.67	6.72
MOON	MLP-80	91.14	7.11	6.72
FedPAQ	MLP-80	55.52	22.24	1.44
SCAFFOLD	MLP-80	did not converge, see text		

TABLE XVI
THE TWO BASELINE CORRECTIONS DO NOT CANCEL; THEY COMPOUND.
TEN SEEDS, PAIRED.

Configuration	KAN-8	MLP-80	Δ
FedAvg, shared η (as submitted)	94.22	88.72	+5.49
FedProx, shared η	94.65	94.92	−0.26
FedProx, own η each	93.85	95.72	−1.87

I. Stronger aggregation removes what is left of the gap

The submitted comparison used FedAvg for both architectures. Since the learning-rate result above shows how much an under-served baseline can distort the conclusion, we ran four further aggregation and compression algorithms on the same cell, ten seeds each: FedProx [28], SCAFFOLD [29], MOON [34] and FedPAQ [7] with top- k sparsification [8]. Reviewer 1 also suggested FedNova [35] and FedDyn [36], and adaptive server optimisation [37] is a further natural comparator; we did not run them and say so rather than add a hasty implementation of each.

FedProx closes the gap, and the two corrections compound. FedProx lifts the parameter-matched MLP by 6.19 pp and cuts its seed variance from 14.21 to 2.67 pp, while lifting FedKAN-8 by 0.44 pp; at the shared step size the difference becomes −0.26 pp in the KAN’s disfavour ($p = 0.82$). A reviewer of an earlier draft pointed out that this table was measured at exactly the step size the rest of the paper argues against, which is a fair objection, so we re-ran it with each architecture at its own best rate. The conclusion does not soften: FedKAN-8 reaches 93.85% and the MLP 95.72%, a difference of −1.87 pp ($p = 0.20$).

The submitted comparison therefore under-served its baseline in two independent ways, a shared learning rate and a FedAvg-only aggregator. Correcting either one alone removes the gap; correcting both puts the parameter-matched MLP ahead. MOON helps neither architecture. FedPAQ at $k = 10\%$ buys a $4.7\times$ uplink reduction at an accuracy neither can afford here: under $\alpha = 0.1$ the client updates already disagree, and discarding nine tenths of each one compounds that.

What we do not claim. Our SCAFFOLD implementation reaches 49.80% for FedKAN-8, which is chance on a binary task, and 73.32% for the MLP. SCAFFOLD’s control variates are derived for local SGD while every experiment here uses

Adam; we believe that is the cause but have not shown it. We report the runs and draw no conclusion about SCAFFOLD. We also note that SCAFFOLD doubles the uplink, 15.38 against 7.69 MB, because each client returns a control variate alongside its model delta: an accounting that charges one model per round per client would make it look free.

What this means for the claim. Scoped to the three NetFlow-v2 datasets and the partitioning regimes we ran, and with every architecture tuned on the same learning-rate grid, the honest statement is: *FedKAN offers no consistent mean-accuracy advantage over a tuned parameter-matched MLP, and no consistent variance advantage; it is markedly less sensitive to the learning rate, at 2.7 to 5.4 times, and markedly more expensive to evaluate, at 19.9 to 37.9 times.* We present this as an empirical observation on three datasets, not as an established property of the architecture class.

This scoping is, we believe, more useful to the community than a universal-superiority claim that the data would not support: it tells the deployer in which regimes FedKAN is most worth adopting (heavily class-imbalanced settings, e.g. Denial-of-Service-focused detection) and where the simpler MLP suffices (more uniformly mixed attack types).

VI. DISCUSSION AND CONCLUSION

A. Summary of Findings

We set out to test whether federated Kolmogorov–Arnold networks are more accurate and more stable than parameter-matched MLPs for NetFlow intrusion detection under client heterogeneity, and on a controlled comparison they are not. Sweeping the learning rate for every architecture over six values, ten seeds and four cells, 1,200 runs in total on a single machine, the mean-accuracy advantage ranges from −0.49 to +6.23 pp with no consistent sign, and on the cell that produced our own headline number it falls from +5.49 to +0.76 pp once the baseline is given its own best step size, and to −0.54 pp ($p = 0.58$) under a nested protocol that chooses the step size on ten seeds and reports on twenty it never saw. The companion variance-reduction claim reverses on three cells of four. Both claims, as stated in the first version of this work, were artefacts of a shared hyper-parameter rather than properties of the architecture.

One property does hold on every dataset we ran. Within a decade-wide band of learning rates, FedKAN-8 loses at most 0.65, 1.58 and 2.53 pp from a bad choice where the parameter-matched MLP loses 3.53, 6.38 and 6.72, a factor of 2.7 to 5.4. That robustness is worth something in a federated deployment where per-client tuning is not available, and it is not free: at matched parameters on single-thread CPU the KAN costs $19.9\times$ more per-sample inference and $37.9\times$ more per batch of 1000, and it transmits more bytes to reach a fixed accuracy target. The contribution of this paper is that trade-off, stated with both of its terms measured, together with a comparison protocol that makes the first term visible: tune every architecture, summarise over several rounds, and report how far the conclusion moves when the summary statistic changes.

B. What the Communication Argument Actually Says

A first generation of FedKAN-IDS work [13] framed the architecture’s appeal in terms of communication efficiency, often citing $\sim 50\%$ bandwidth reductions. Proposition 2 and a parameter-matched protocol show that framing is artefactual: the savings reflect an oversized MLP baseline, not a property of the KAN. Under parameter parity the direction reverses, and we report the measured per-round uplink alongside the counts that Proposition 2 predicts from them so that the reader can check the arithmetic rather than take our word for it.

We do not offer a replacement value proposition based on tail risk. An earlier version of this paper did, citing a +24.24 pp worst-seed advantage on NF-BoT-IoT-v2. That figure is real but it is one of three: the same comparison gives -11.07 pp on NF-ToN-IoT-v2 and -21.23 pp on NF-CSE-CIC-IDS2018-v2, both against the KAN. Quoting the favourable third and describing it as a credible architectural contribution is not a reframing, it is a selection, and we withdraw it.

C. Threats to Validity

Learning rates were selected on the evaluation seeds. Each architecture’s best step size was chosen by comparing mean accuracy over the same ten seeds used for the reported comparison. This inflates both architectures, and it inflates the one with more spread across the grid more. A protocol with a held-out tuning split would give smaller absolute numbers for both; we did not run one, and the tuned figures should be read as optimistic for both sides.

The MLP’s optimum sits at the edge of the grid. On NF-BoT-IoT-v2 the parameter-matched MLP peaks at $\eta = 10^{-1}$, the largest value we swept. The correction it produces, from +5.49 to +0.76 pp, is therefore a *lower bound* on the confound: a wider grid could shrink the residual further, or reverse it. We report the endpoint rather than present the residual as settled.

Theorem 1 is one-sided. The bound is stated for the KAN and no corresponding bound is derived for the MLP, so it cannot license a comparative claim in either direction. Its additive spline term is non-negative, which if anything predicts a *worse* optimisation floor for the KAN. We retain the theorem for what it does establish, that the spline bias enters additively and does not scale with heterogeneity, and we no longer cite it in support of any comparison.

Unit of analysis. Section V-G reports the pooled comparison under both a seed-pair and a cell unit because they answer different questions. The per-cell tables elsewhere in Section V use seed-paired tests, which are appropriate for a statement about that cell and not for a statement about the architectures in general.

Four cells is a small sample for a general claim. A single cell accounts for 74% of the between-cell variance of the tuned advantage, and we have not run a leave-one-cell-out check.

Static grid hyperparameters in the main comparison. The headline comparison fixes $G = 5, k = 3$. Section V-G varies both, and $G = 10$ is better on every metric we report while costing 50% more parameters, which breaks the parity the comparison rests on.

D. Future Work

Three concrete extensions present themselves. First, combining FedKAN with grid sparsification along the lines of CG-FKAN [18] should let the architecture’s robustness benefit be paid for with reduced rather than increased communication. Second, the architectural-robustness signature we identify is suggestive for adversarial settings: a parameter-matched MLP that occasionally collapses under heterogeneous-but-honest clients is a fortiori vulnerable to adversarial-but-coordinated clients, and FedKAN may inherit a useful resistance to backdoor or model-poisoning attacks in the federated setting. Third, the small parameter footprint of FedKAN-8 (~ 13 kB float32, ~ 3.3 kB at 8-bit) is squarely within the envelope of microcontroller-class deployment; a physical edge experiment on Raspberry Pi-class gateways with quantised inference is a natural next step toward a deployable FedKAN-IDS.

E. Reproducibility

All code, dataset preparation scripts, configuration files, per-run metrics for the 1,500 reproducible runs on which the paper’s numbers rest reported in Section V, the auto-generated tables and figures of this manuscript, and the $\text{\LaTeX}$ source of the manuscript itself are released at <https://github.com/haodpsut/FedKAN-IDS-v2>. Experiments were executed on an NVIDIA RTX 4090 workstation managed by the conda environment recipe in docs/SERVER_SETUP.md; a one-line invocation `bash scripts/run_grid.sh` reproduces any of the three datasets, and `bash scripts/rebuild_paper_artifacts.sh` regenerates every table, figure, and the compiled PDF from the committed run directories. A Google Colab notebook is also included in notebooks/ for users without a dedicated GPU, but all reported wall-clock numbers in this paper refer to the RTX 4090 setup.

ACKNOWLEDGMENT

The authors used Anthropic’s Claude (Opus 5) as an assistant during the revision of this manuscript. Its use covered three areas: drafting and editing prose in the introduction and in Sections V and VI; writing the analysis and experiment-orchestration scripts released with the artifact; and proposing the learning-rate sweep, the sensitivity grid and the Hessian-spectrum measurement reported in Sections IV and V. All experimental results were produced by executing that released code, and every number reported here was checked against the run records by the authors, who take full responsibility for the content of this paper.

REFERENCES

- [1] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, “Federated learning for internet of things: A comprehensive survey,” *IEEE communications surveys & tutorials*, vol. 23, no. 3, pp. 1622–1658, 2021.
- [2] P. H. Do, T. D. Le, V. Vishnevsky, A. Berezkin, and R. Kirichek, “A horizontal federated learning approach to iot malware traffic detection: An empirical evaluation with n-baiot dataset,” in *2024 26th International Conference on Advanced Communications Technology (ICACT)*, 2024, pp. 1494–1506.

- [3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. Pmlr, 2017, pp. 1273–1282.
- [4] Z. Lu, H. Pan, Y. Dai, X. Si, and Y. Zhang, "Federated learning with non-iid data: A survey," *IEEE Internet of Things Journal*, vol. 11, no. 11, pp. 19 188–19 209, 2024.
- [5] B. Li, Y. Wu, J. Song, R. Lu, T. Li, and L. Zhao, "Deepfed: Federated deep learning for intrusion detection in industrial cyber-physical systems," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 8, pp. 5615–5624, 2020.
- [6] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-iid data," *IEEE transactions on neural networks and learning systems*, vol. 31, no. 9, pp. 3400–3413, 2019.
- [7] A. Reisizadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, and R. Pedarsani, "Fedpaq: A communication-efficient federated learning method with periodic averaging and quantization," in *International conference on artificial intelligence and statistics*. PMLR, 2020, pp. 2021–2031.
- [8] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proceedings of the 2017 conference on empirical methods in natural language processing*, 2017, pp. 440–445.
- [9] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.
- [10] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljagic, T. Hou, and M. Tegmark, "Kan: Kolmogorov-arnold networks," in *International conference on learning representations*, vol. 2025, 2025, pp. 70 367–70 413.
- [11] P. Hao Do, T. Duc Le, T. Duy Dinh, and v. Dai Pham, "Classifying iot botnet attacks with kolmogorov-arnold networks: A comparative analysis of architectural variations," *IEEE Access*, vol. 13, pp. 16 072–16 093, 2025.
- [12] E. Zeydan, C. J. Vaca-Rubio, L. Blanco, R. Pereira, M. Caus, and A. Aydeger, "F-kans: Federated kolmogorov-arnold networks," in *2025 IEEE 22nd Consumer Communications & Networking Conference (CCNC)*. IEEE, 2025, pp. 1–6.
- [13] A. M. Sasse and C. M. de Farias, "Evaluating federated kolmogorov-arnold networks on non-iid data," *arXiv preprint arXiv:2410.08961*, 2024.
- [14] Y. Ma, Z. Yang, and L.-D. Ibáñez, "Enhancing federated learning with kolmogorov-arnold networks: A comparative study across diverse aggregation strategies," *arXiv preprint arXiv:2505.07629*, 2025.
- [15] E. Zeydan, C. J. Vaca-Rubio, L. Blanco, R. Pereira, M. Caus, and K. Dev, "Fed-kan: Federated learning with kolmogorov-arnold networks for traffic prediction," *arXiv preprint arXiv:2503.00154*, 2025.
- [16] Y. Lee, J. Gong, and J. Kang, "A unified benchmark of federated learning with kolmogorov-arnold networks for medical imaging," *arXiv preprint arXiv:2504.19639*, 2025.
- [17] S. N. Zeleke and M. Boichichio, "Federated kolmogorov-arnold networks for health data analysis: A study using ecg signal," in *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 2024, pp. 8070–8077.
- [18] S. Yu, Y. Lee, J. Gong, and J. Kang, "Cg-fkan: Compressed-grid federated kolmogorov-arnold networks for communication constrained environment," *IEEE Communications Letters*, 2026.
- [19] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile networks and applications*, vol. 27, no. 1, pp. 357–370, 2022.
- [20] G. Melis, C. Dyer, and P. Blunsom, "On the state of the art of evaluation in neural language models," in *Int. Conf. Learn. Represent. (ICLR)*, 2018, arXiv:1707.05589.
- [21] K. Musgrave, S. Belongie, and S.-N. Lim, "A metric learning reality check," in *Eur. Conf. Comput. Vis. (ECCV)*, 2020, arXiv:2003.08505.
- [22] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are we really making much progress? a worrying analysis of recent neural recommendation approaches," in *ACM Conf. Recommender Systems (RecSys)*, 2019.
- [23] M. Lucic, K. Kurach, M. Michalski, S. Gelly, and O. Bousquet, "Are GANs created equal? a large-scale study," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2018, arXiv:1711.10337.
- [24] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *AAAI Conf. Artif. Intell.*, 2018, arXiv:1709.06560.
- [25] J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith, "Show your work: Improved reporting of experimental results," in *Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, 2019.
- [26] X. Bouthillier, P. Delaunay, M. Bronzi *et al.*, "Accounting for variance in machine learning benchmarks," in *Proc. Mach. Learn. Syst. (MLSys)*, 2021, arXiv:2103.03098.
- [27] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, 2021, arXiv:1912.04977.
- [28] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst. (MLSys)*, 2020, arXiv:1812.06127.
- [29] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Int. Conf. Mach. Learn. (ICML)*, 2020, arXiv:1910.06378.
- [30] L. Schumaker, *Spline functions: basic theory*. Cambridge university press, 2007.
- [31] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of fedavg on non-iid data," *arXiv preprint arXiv:1907.02189*, 2019.
- [32] B. Ghorbani, S. Krishnan, and Y. Xiao, "An investigation into neural net optimization via hessian eigenvalue density," in *Int. Conf. Mach. Learn. (ICML)*, 2019, arXiv:1901.10159.
- [33] Z. Yao, A. Gholami, K. Keutzer, and M. W. Mahoney, "PyHessian: Neural networks through the lens of the Hessian," in *IEEE Int. Conf. Big Data*, 2020.
- [34] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, arXiv:2103.16257.
- [35] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, arXiv:2007.07481.
- [36] D. A. E. Acar, Y. Zhao, R. Matas, M. Mattina, P. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," in *Int. Conf. Learn. Represent. (ICLR)*, 2021, arXiv:2111.04263.
- [37] S. J. Reddi, Z. Charles, M. Zaheer *et al.*, "Adaptive federated optimization," in *Int. Conf. Learn. Represent. (ICLR)*, 2021, arXiv:2003.00295.